CSC3100: MinHash & Locality Sensitive Hashing

Tianshu Yu

2024 Summer

Goals

Many Web-mining problems can be expressed as finding "similar" sets:

1. Pages with similar words, e.g., for classification by topic.
2. NetFlix users with similar tastes in movies, for recommendation systems.
3. Dual: movies with similar sets of fans.
4. Images of related things.

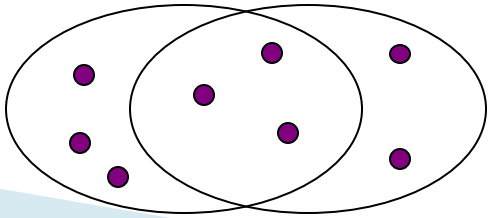
Motivating problem

- Find duplicate and near-duplicate documents from a web crawl.

- If we wanted exact duplicates we could do this by hashing
  - We will see how to adapt this technique for near duplicate documents

Distance Measures

- **Goal: Find near-neighbors in high-dim. space**
  - We formally define "near neighbors" as points that are a "small distance" apart
- For each application, we first need to define what "**distance**" means
- **Today: Jaccard distance/similarity**
  - The **Jaccard similarity** of two **sets** is the size of their intersection divided by the size of their union: $sim(C_1, C_2) = |C_1 \cap C_2| / |C_1 \cup C_2|$
  - **Jaccard distance:** $d(C_1, C_2) = 1 - |C_1 \cap C_2| / |C_1 \cup C_2|$



3 in intersection
8 in union
Jaccard similarity= 3/8
Jaccard distance = 5/8

Task: Finding Similar Documents

- **Goal: Given a large number (N in the millions or billions) of documents, find "near duplicate" pairs**
- **Applications:**
  - Mirror websites, or approximate mirrors
    - Don't want to show both in search results
  - Similar news articles at many news sites
    - Cluster articles by "same story"

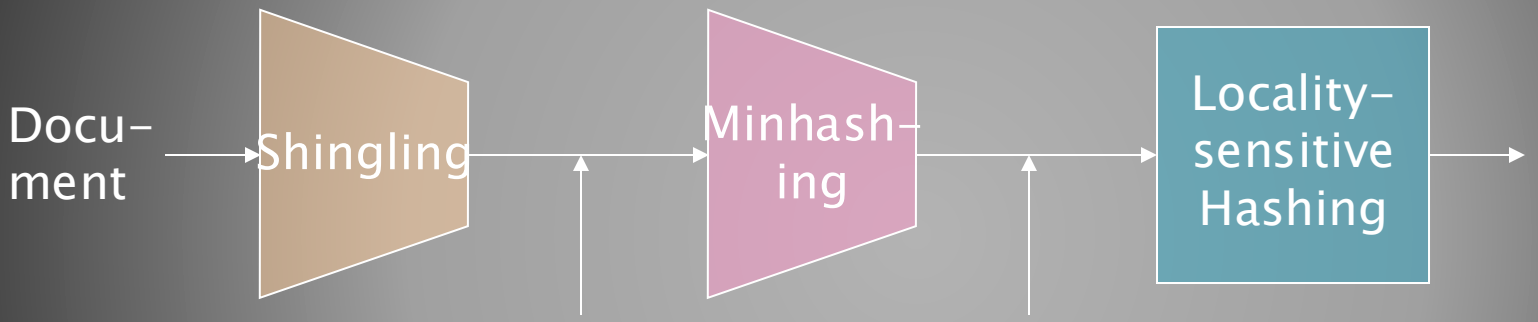
Main issues

- What is the right representation of the document when we check for similarity?
  - E.g., representing a document as a set of characters will not do (why?)
- When we have billions of documents, keeping the full text in memory is not an option.
  - We need to find a shorter representation
- How do we do pairwise comparisons of billions of documents?
  - If exact match was the issue it would be ok, can we replicate this idea?

3 Essential Steps for Similar Docs

1. *Shingling:* Convert documents to sets
2. *Min-Hashing:* Convert large sets to short signatures, while preserving similarity
3. *Locality-Sensitive Hashing:* Focus on pairs of signatures likely to be from similar documents
   - **Candidate pairs!**

The Big Picture



The set of strings of length *k* that appear in the doc-ument

Signatures : short integer vectors that represent the sets, and reflect their similarity

Candidate pairs : those pairs of signatures that we need to test for similarity.

Documents as High-Dim Data

- **Step 1: *Shingling:* Convert documents to sets**
- **Simple approaches:**
  - Document = set of words appearing in document
  - Document = set of "important" words
  - Don't work well for this application. Why?
- **Need to account for ordering of words!**
- A different way: **Shingles!**

Define: Shingles

- A *k*-shingle (or *k*-gram) for a document is a sequence of *k* tokens that appears in the doc
  - Tokens can be characters, words or something else, depending on the application
  - Assume tokens = characters for examples

- **Example:** k=2; document D_1 = abcab
  Set of 2-shingles: $S(D_1)$ = {ab, bc, ca}
  - **Option:** Shingles as a bag (multiset), count ab twice: $S'(D_1)$ = {ab, bc, ca, ab}

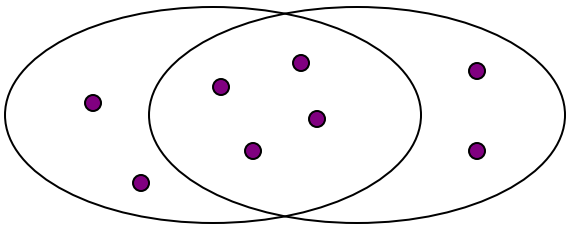
Compressing Shingles

- To **compress long shingles**, we can **hash** them to (say) 4 bytes
- **Represent a document by the set of hash values of its *k*-shingles**
  - **Idea:** Two documents could (rarely) appear to have shingles in common, when in fact only the hash-values were shared
- **Example:** k=2; document $D_1 =$ abcab
  Set of 2-shingles: $S(D_1) = \{ab, bc, ca\}$
  Hash the singles: $h(D_1) = \{1, 5, 7\}$

Similarity Metric for Shingles

- **Document D_1 is a set of its k-shingles $C_1=S(D_1)$**
- Equivalently, each document is a 0/1 vector in the space of *k*-shingles
  - Each unique shingle is a dimension
  - Vectors are very sparse
- **A natural similarity measure is the Jaccard similarity:**

$$sim(D_1, D_2) = |C_1 \cap C_2| / |C_1 \cup C_2|$$



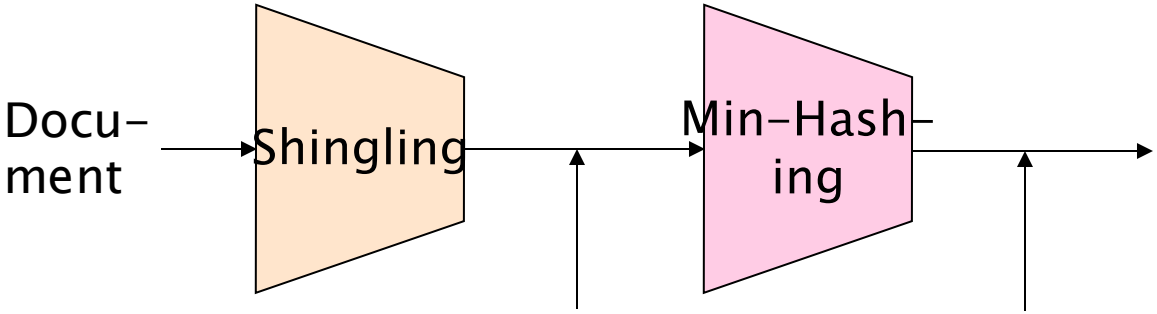
Working Assumption

- **Documents that have lots of shingles in common have similar text, even if the text appears in different order**

- **Caveat:** You must pick ***k*** large enough, or most documents will have most shingles
  - ***k*** = 5 is OK for short documents
  - ***k*** = 10 is better for long documents

Motivation for Minhash/LSH

- **Suppose we need to find near-duplicate documents among $N = 1$ million documents**
- Naïvely, we would have to compute **pairwise Jaccard similarities** for **every pair of docs**
  - $\boldsymbol{N(N-1)/2 \approx 5*10^{11}}$ comparisons
  - At 10^5 secs/day and 10^6 comparisons/sec, it would take **5 days**
- For $\boldsymbol{N = 10}$ million, it takes more than a year…



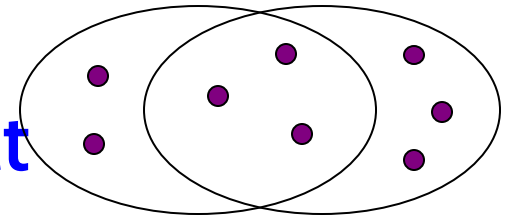
The set of strings of length *k* that appear in the document

Signatures: short integer vectors that represent the sets, and reflect their similarity

MinHashing

Step 2: ***Minhashing:*** Convert large sets to short signatures, while preserving similarity

Encoding Sets as Bit Vectors



- Many similarity problems can be formalized as **finding subsets that have significant intersection**
- **Encode sets using 0/1 (bit, boolean) vectors**
  - One dimension per element in the universal set
- Interpret set intersection as bitwise **AND**, and
  set union as bitwise **OR**

- **Example:** $C_1 = 10111$; $C_2 = 10011$
  - Size of intersection = **3**; size of union = **4**,
  - **Jaccard similarity** (not distance) = **3/4**
  - **Distance: $d(C_1,C_2)$ = 1 – (Jaccard similarity) = 1/4**

From Sets to Boolean Matrices

- **Rows** = elements (shingles)
- **Columns** = sets (documents)
  - 1 in row ***e*** and column ***s*** if and only if ***e*** is a member of ***s***
  - Column similarity is the Jaccard similarity of the corresponding sets (rows with value *1)*
  - **Typical matrix is sparse!**
- **Each document is a column:**
  - **Example: sim(C_1 ,C_2) = ?**
    - Size of intersection = 3; size of union = 6,
      Jaccard similarity (not distance) = 3/6
    - **d(C_1, C_2) = 1 – (Jaccard similarity) = 3/6**

Documents

Shingles

1	1	1	0
1	1	0	1
0	1	0	1
0	0	0	1
1	0	0	1
1	1	1	0
1	0	1	0

Outline: Finding Similar Columns

- **So far:**
  - Documents → Sets of shingles
  - Represent sets as boolean vectors in a matrix
- **Next goal: Find similar columns while computing small signatures**
  - **Similarity of columns == similarity of signatures**

Outline: Finding Similar Columns

- **Next Goal: Find similar columns, Small signatures**
- **Naïve approach:**
  - **1) Signatures of columns:** small summaries of columns
  - **2) Examine pairs of signatures** to find similar columns
    - **Essential:** Similarities of signatures and columns are related
  - **3) Optional:** Check that columns with similar signatures are really similar
- **Warnings:**
  - Comparing all pairs may take too much time: **Job for LSH**
    - These methods can produce false negatives, and even false positives (if the optional check is not made)

Hashing Columns (Signatures)

- **Key idea:** "hash" each column ***C*** to a small *signature* ***h(C)***, such that:
  - (1) ***h(C)*** is small enough that the signature fits in RAM
  - (2) ***sim(C_1, C_2)*** is the same as the "similarity" of signatures ***h(C_1)*** and ***h(C_2)***

- **Goal: Find a hash function *h(·)* such that:**
  - If ***sim(C_1,C_2)*** is high, then with high prob. ***h(C_1) = h(C_2)***
  - If ***sim(C_1,C_2)*** is low, then with high prob. ***h(C_1) ≠ h(C_2)***

- **Hash docs into buckets. Expect that "most" pairs of near duplicate docs hash into the same bucket!**

Min-Hashing

- **Goal: Find a hash function $h(\cdot)$ such that:**
  - if $sim(C_1, C_2)$ is high, then with high prob. $h(C_1) = h(C_2)$
  - if $sim(C_1, C_2)$ is low, then with high prob. $h(C_1) \neq h(C_2)$

- **Clearly, the hash function depends on the similarity metric:**
  - Not all similarity metrics have a suitable hash function
- **There is a suitable hash function for the Jaccard similarity:** It is called **Min-Hashing**

Min-Hashing

- Imagine the rows of the boolean matrix permuted under **random permutation** π
- Define a **"hash" function** $h_\pi(C)$ = the index of the **first** (in the permuted order π) row in which column C has value 1:

$$h_\pi(C) = min_\pi \pi(C)$$

- Use several (e.g., 100) independent hash functions (that is, permutations) to create a signature of a column

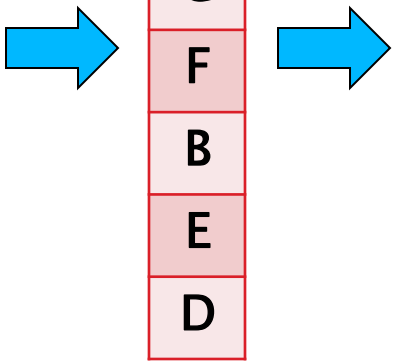
Min-Hashing

- Pick a random permutation of the rows (the universe U).
- Define "hash" function for set S
  - h(S) = the index of the first row (in the permuted order) in which column S has 1.
  - OR
  - h(S) = the index of the first element of S in the permuted order.
- Use k (e.g., k = 100) independent random permutations to create a signature.

Example of minhash signatures

- Input matrix

	S_1	S_2	S_3	S_4
A	1	0	1	0
B	1	0	0	1
C	0	1	0	1
D	0	1	0	1
E	0	1	0	1
F	1	0	1	0
G	1	0	1	0

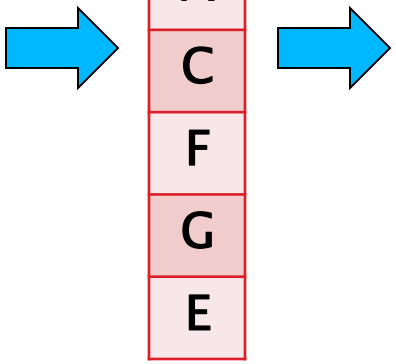


		S_1	S_2	S_3	S_4
1	A	1	0	1	0
2	C	0	1	0	1
3	G	1	0	1	0
4	F	1	0	1	0
5	B	1	0	0	1
6	E	0	1	0	1
7	D	0	1	0	1
		1	2	1	2

Example of minhash signatures

- Input matrix

	S_1	S_2	S_3	S_4
A	1	0	1	0
B	1	0	0	1
C	0	1	0	1
D	0	1	0	1
E	0	1	0	1
F	1	0	1	0
G	1	0	1	0

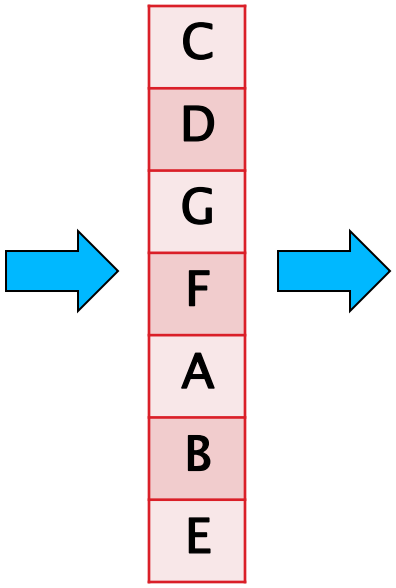


		S_1	S_2	S_3	S_4
1	D	0	1	0	1
2	B	1	0	0	1
3	A	1	0	1	0
4	C	0	1	0	1
5	F	1	0	1	0
6	G	1	0	1	0
7	E	0	1	0	1
		2	1	3	1

Example of minhash signatures

- Input matrix

	S_1	S_2	S_3	S_4
A	1	0	1	0
B	1	0	0	1
C	0	1	0	1
D	0	1	0	1
E	0	1	0	1
F	1	0	1	0
G	1	0	1	0



		S_1	S_2	S_3	S_4
1	C	0	1	0	1
2	D	0	1	0	1
3	G	1	0	1	0
4	F	1	0	1	0
5	A	1	0	1	0
6	B	1	0	0	1
7	E	0	1	0	1
		3	1	3	1

Example of minhash signatures

- Input matrix

	S_1	S_2	S_3	S_4
A	1	0	1	0
B	1	0	0	1
C	0	1	0	1
D	0	1	0	1
E	0	1	0	1
F	1	0	1	0
G	1	0	1	0

≈

Signature matrix

	S_1	S_2	S_3	S_4
h_1	1	2	1	2
h_2	2	1	3	1
h_3	3	1	3	1

- Sig(S) = vector of hash values
  - e.g., Sig(S_2) = [2,1,1]
- Sig(S,i) = value of the i-th hash function for set S
  - E.g., Sig(S_2,3) = 1

Hash function Property

$$Pr(h(S_1) = h(S_2)) = Sim(S_1,S_2)$$

- where the probability is over all choices of permutations.

- Why?
  - The first row where one of the two sets has value 1 belongs to the union.
    - Recall that union contains rows with at least one 1.
  - We have equality if both sets have value 1, and this row belongs to the intersection

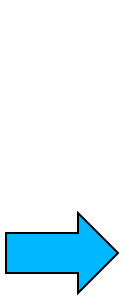
Example

- Universe: U = {A,B,C,D,E,F,G}
- X = {A,B,F,G}
- Y = {A,E,F,G}

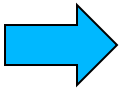
- Union = {A,B,E,F,G}
- Intersection = {A,F,G}

Rows C,D could be anywhere they do not affect the probability

	X	Y
A	1	1
B	1	0
C	0	0
D	0	0
E	0	1
F	1	1
G	1	1



D
*
*
C
*
*
*



	X	Y
D	0	0
C	0	0

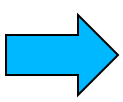
Example

- Universe: U = {A,B,C,D,E,F,G}
- X = {A,B,F,G}
- Y = {A,E,F,G}

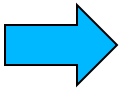
- Union = {A,B,E,F,G}
- Intersection = {A,F,G}

The * rows belong to the union

	X	Y
A	1	1
B	1	0
C	0	0
D	0	0
E	0	1
F	1	1
G	1	1



D
*
*
C
*
*
*



	X	Y
D	0	0
C	0	0

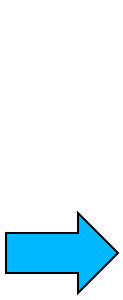
Example

- Universe: U = {A,B,C,D,E,F,G}
- X = {A,B,F,G}
- Y = {A,E,F,G}

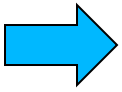
- Union = {A,B,E,F,G}
- Intersection = {A,F,G}

The question is what is the value of the first * element

	X	Y
A	1	1
B	1	0
C	0	0
D	0	0
E	0	1
F	1	1
G	1	1



D
*
*
C
*
*
*



	X	Y
D	0	0
C	0	0

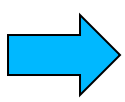
Example

- Universe: U = {A,B,C,D,E,F,G}
- X = {A,B,F,G}
- Y = {A,E,F,G}

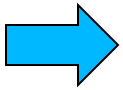
If it belongs to the intersection then h(X) = h(Y)

- Union = {A,B,E,F,G}
- Intersection = {A,F,G}

	X	Y
A	1	1
B	1	0
C	0	0
D	0	0
E	0	1
F	1	1
G	1	1



D
*
*
C
*
*
*



	X	Y
D	0	0
C	0	0

Example

- Universe: U = {A,B,C,D,E,F,G}
- X = {A,B,F,G}
- Y = {A,E,F,G}

Every element of the union is equally likely to be the * element

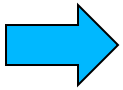
$$\Pr(h(X) = h(Y)) = \frac{|\{A,F,G\}|}{|\{A,B,E,F,G\}|} = \frac{3}{5} = \text{Sim}(X,Y)$$

- Union = {A,B,E,F,G}
- Intersection = {A,F,G}

	X	Y
A	1	1
B	1	0
C	0	0
D	0	0
E	0	1
F	1	1
G	1	1



D
*
*
C
*
*
*



	X	Y
D	0	0
C	0	0

Similarity for Signatures

- The similarity of signatures is the fraction of the hash functions in which they agree.

	S_1	S_2	S_3	S_4
A	1	0	1	0
B	1	0	0	1
C	0	1	0	1
D	0	1	0	1
E	0	1	0	1
F	1	0	1	0
G	1	0	1	0

≈

Signature matrix

S_1	S_2	S_3	S_4
1	2	1	2
2	1	3	1
3	1	3	1

	Actual	Sig
(S_1, S_2)	0	0
(S_1, S_3)	3/5	2/3
(S_1, S_4)	1/7	0
(S_2, S_3)	0	0
(S_2, S_4)	3/4	1
(S_3, S_4)	0	0

Zero similarity is preserved
High similarity is well approximated

- With multiple signatures we get a good approximation

Is it now feasible?

- Assume a billion rows
- Hard to pick a random permutation of 1…billion
- **Even representing a random permutation requires 1 billion entries!!!**
- How about accessing rows in permuted order? ☹

Being more practical

- Instead of permuting the rows we will apply a hash function that maps the rows to a new (possibly larger) space
  - The value of the hash function is the position of the row in the new order (permutation).
  - Each set is represented by the smallest hash value among the elements in the set
- The space of the hash functions should be such that if we select one at random, each element (row) has equal probability to have the smallest value
  - Min-wise independent hash functions

Algorithm – One set, one hash function

Computing **Sig(S,i)** for a single column S and single hash function h_i

In practice only the rows (shingles) that appear in the data

for each row **r**

compute h_i (**r**)

h_i (**r**) = index of row r in permutation

if column **S** that has **1** in row **r**

S contains row r

if h_i (**r**) is a smaller value than **Sig(S,i)** **then**

Sig(S,i) = h_i (**r**);

Find the row r with minimum index

Sig(S,i) will become the smallest value of h_i(**r**) among all rows (shingles) for which column **S** has value **1** (shingle belongs in S)*; i.*e., h_i (**r**) gives the min index for the **i**–th permutation

How to pick a random hash function h(x)?

Universal hashing:

- $h_{a,b}(x)=((a \cdot x+b) \bmod p) \bmod N$

where:

- a,b … random integers
- p … prime number (p > N)

Algorithm – All sets, k hash functions

Pick k=100 hash functions ($h_1,\ldots,h_k$)

```
for each row r
    for each hash function h_i
        compute h_i (r )
        for each column S that has 1 in row r
            if h_i (r ) is a smaller value than Sig(S,i) then
                Sig(S,i) = h_i (r);
```

In practice this means selecting the hash function parameters

Compute $h_i(r)$ only once for all sets

Example

x	Row	S1	S2	h(x)	g(x)
0	A	1	0	1	3
1	B	0	1	2	0
2	C	1	1	3	2
3	D	1	0	4	4
4	E	0	1	0	1

$h(x) = x+1 \bmod 5$
$g(x) = 2x+3 \bmod 5$

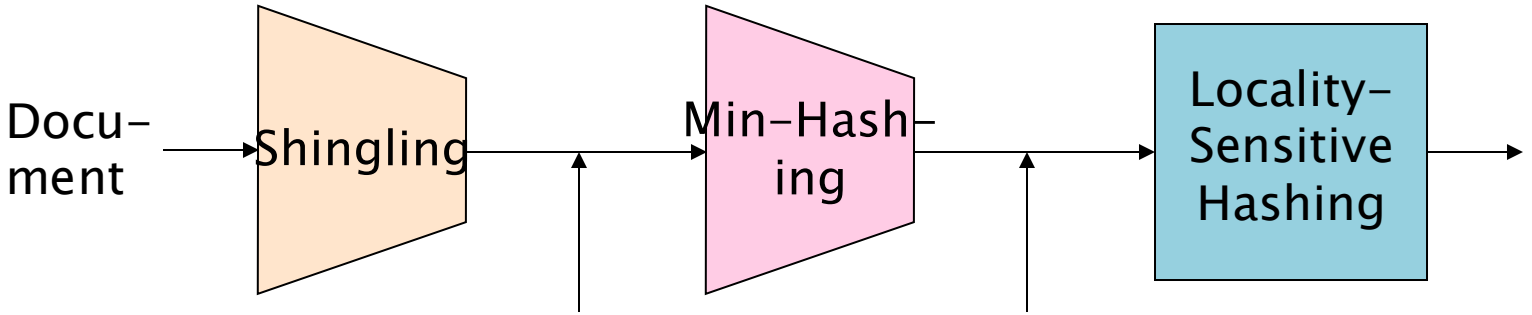
h(Row)	Row	S1	S2
0	E	0	1
1	A	1	0
2	B	0	1
3	C	1	1
4	D	1	0

g(Row)	Row	S1	S2
0	B	0	1
1	E	0	1
2	C	1	0
3	A	1	1
4	D	1	0

	Sig1	Sig2
$h(0) = 1$	1	–
$g(0) = 3$	3	–
$h(1) = 2$	1	2
$g(1) = 0$	3	0
$h(2) = 3$	1	2
$g(2) = 2$	2	0
$h(3) = 4$	1	2
$g(3) = 4$	2	0
$h(4) = 0$	1	0
$g(4) = 1$	2	0

Implementation

- Often, data is given by column, not row.
  - E.g., columns = documents, rows = shingles.
- If so, sort matrix once so it is by row.
- And always compute $h_i(r)$ only once for each row.



Step 3: *Locality-Sensitive Hashing:*

Focus on pairs of signatures likely to be from similar documents

LSH: First Cut

- **Goal:** Find documents with Jaccard similarity at least ***s*** (for some similarity threshold, e.g., ***s***=0.8)

- **LSH – General idea:** Use a function ***f(x,y)*** that tells whether ***x*** and ***y*** is a ***candidate pair:*** a pair of elements whose similarity must be evaluated

- **For Min-Hash matrices:**
  - Hash columns of signature matrix ***M*** to many buckets
  - Each pair of documents that hashes into the same bucket is a **candidate pair**

Candidates from Min-Hash

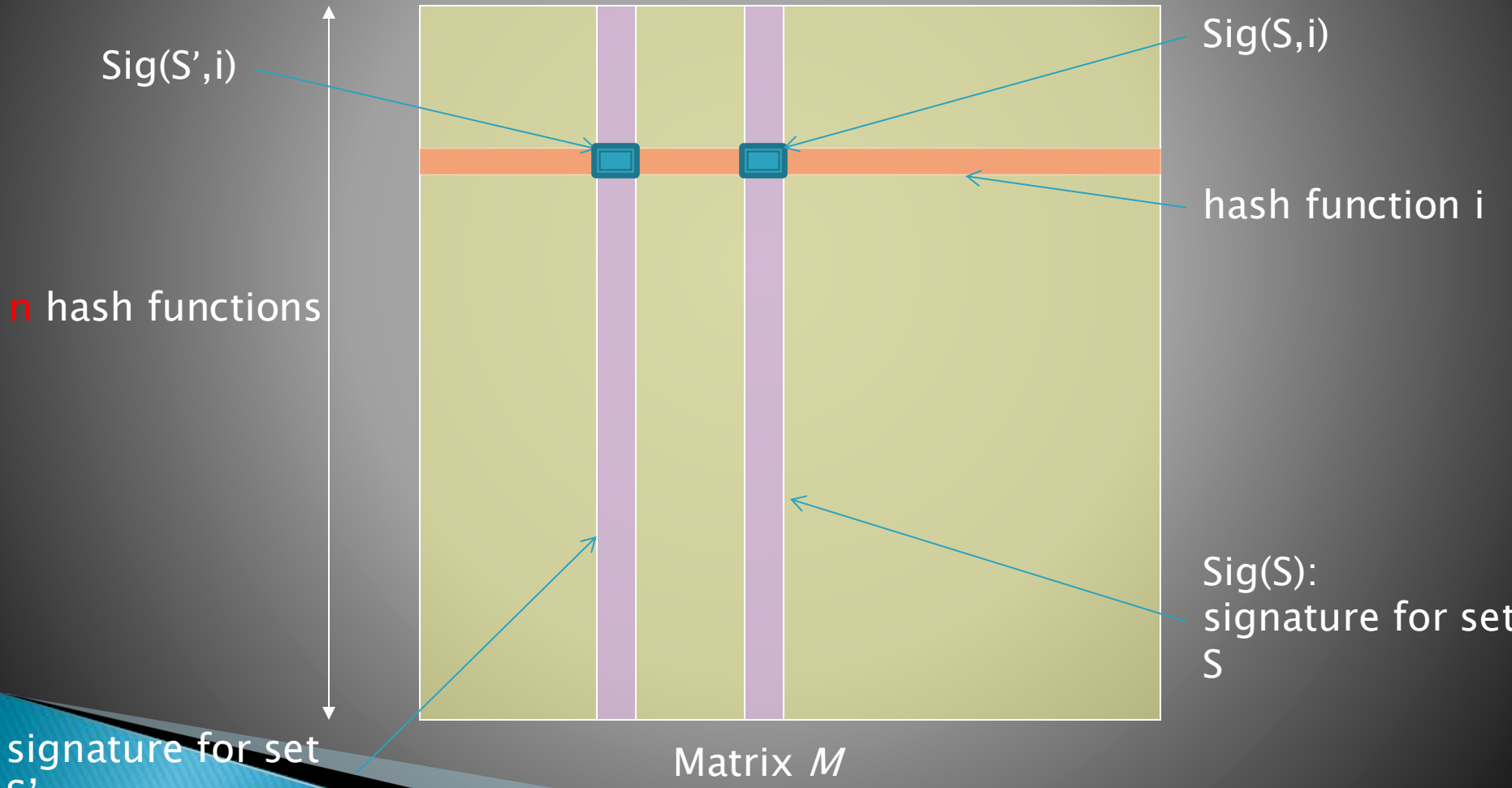
- **Pick a similarity threshold s ($0 < s < 1$)**
- Columns x and y of M are a **candidate pair** if their signatures agree on at least fraction s of their rows:
  $M(i, x) = M(i, y)$ for at least frac. s values of i
  - We expect documents x and y to have the same (Jaccard) similarity as their signatures

LSH for Min-Hash

- **Big idea: Hash columns of signature matrix *M* several times**
- Arrange that (only) **similar columns** are likely to **hash to the same bucket**, with high probability
- **Candidate pairs are those that hash to the same bucket**

Signature matrix reminder

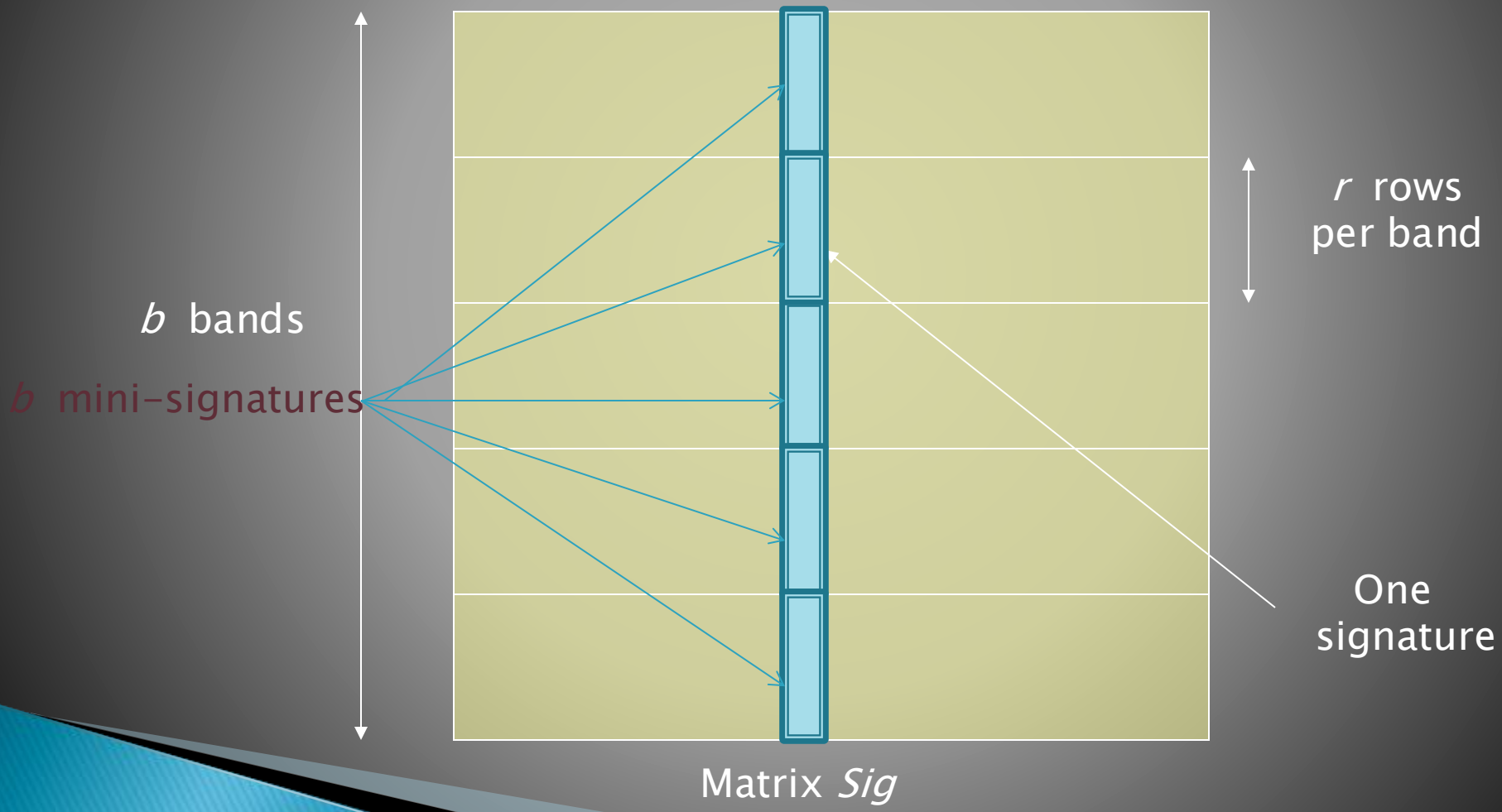
Prob(Sig(S,i) == Sig(S',i)) = sim(S,S')



Partition into Bands – (1)

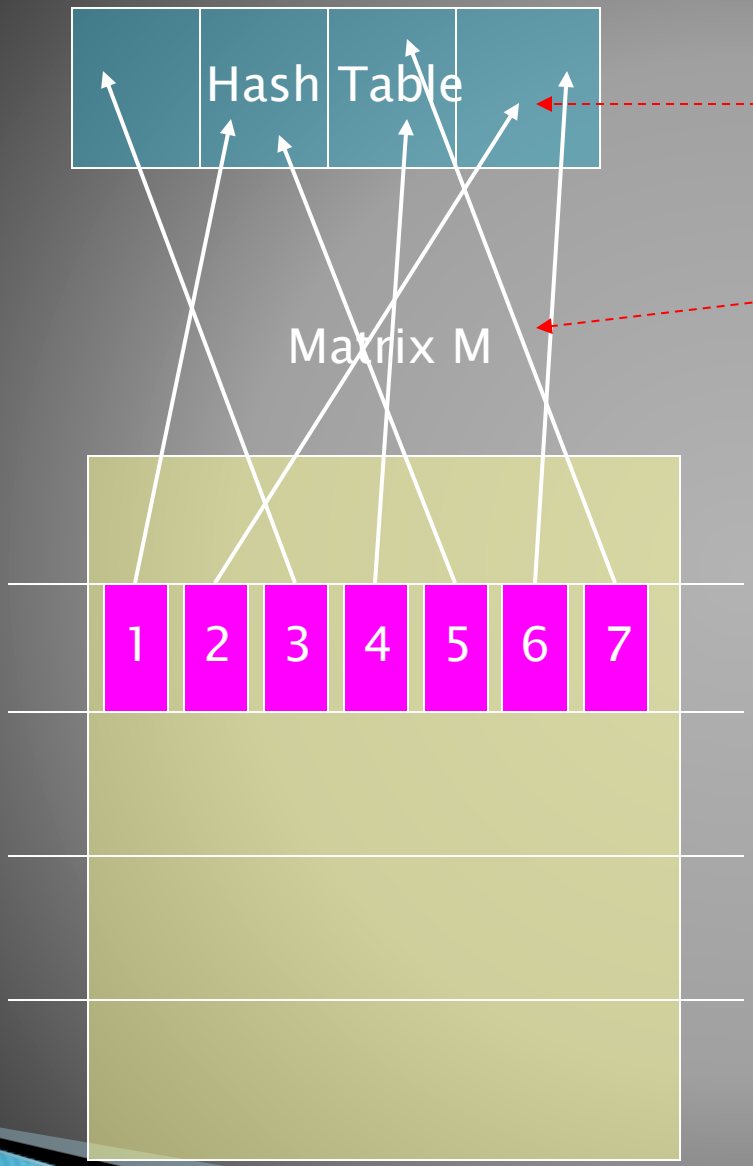
- Divide the signature matrix Sig into *b* bands of *r* rows.
  - Each band is a mini-signature with r hash functions.

Partitioning into bands



Partition into Bands – (2)

- Divide the signature matrix Sig into *b* bands of *r* rows.
  - Each band is a mini-signature with r hash functions.
- For each band, hash the mini-signature to a hash table with *k* buckets.
  - Make *k* as large as possible so that mini-signatures that hash to the same bucket are almost certainly identical.



Columns 2 and 6
are (almost certainly) identical.

Columns 6 and 7 are
surely different.

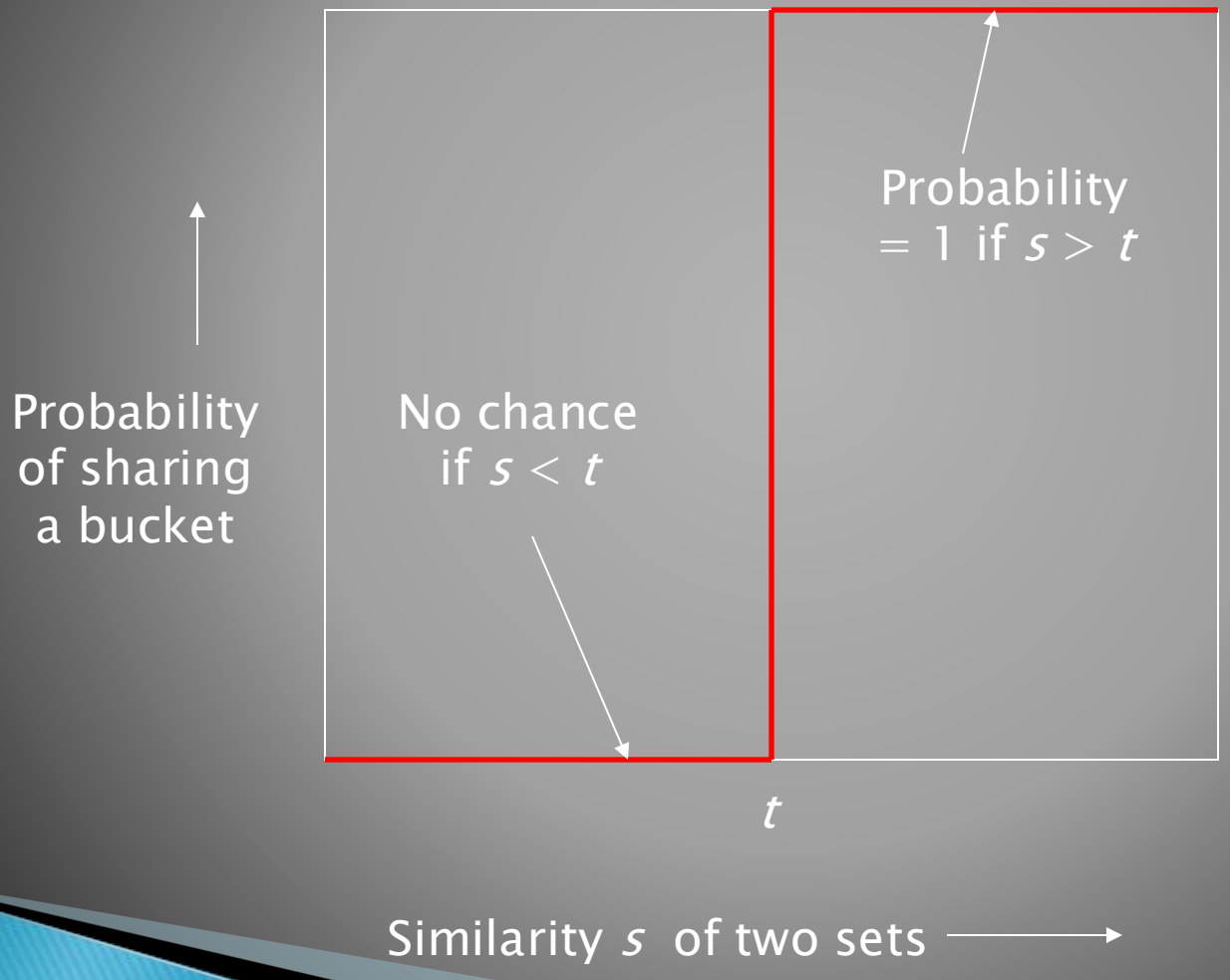
r rows

b bands

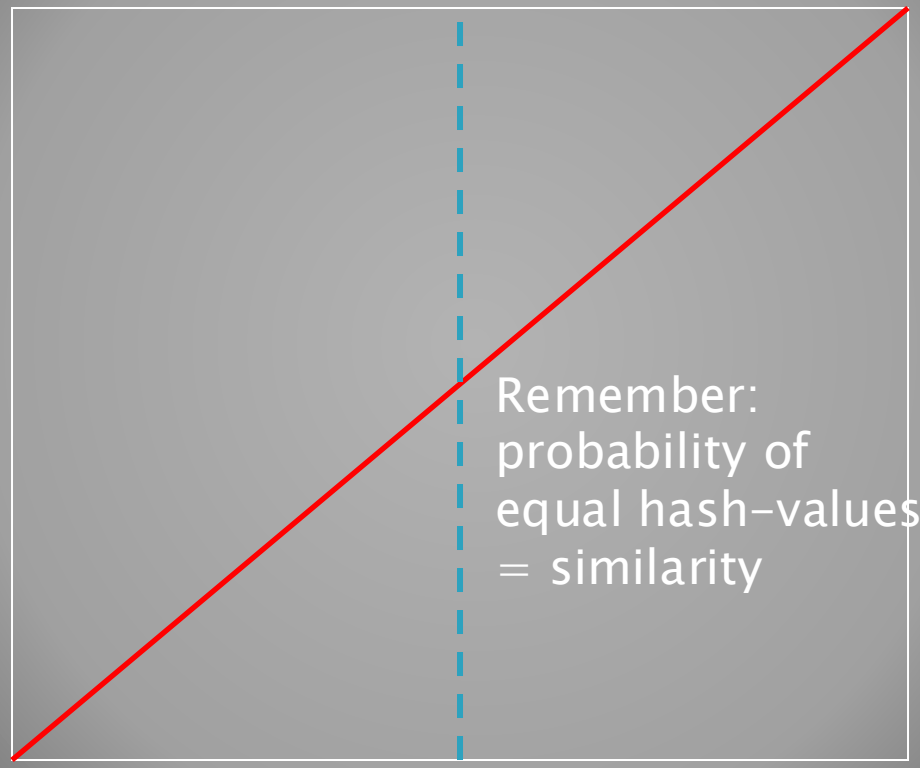
Partition into Bands – (3)

- Divide the signature matrix Sig into *b* bands of *r* rows.
  - Each band is a mini-signature with r hash functions.
- For each band, hash the mini-signature to a hash table with *k* buckets.
  - Make *k* as large as possible so that mini-signatures that hash to the same bucket are almost certainly identical.
- Candidate column pairs are those that hash to the same bucket for at least 1 band.
- Tune *b* and *r* to catch most similar pairs, but few non-similar pairs.

Analysis of LSH – What We Want

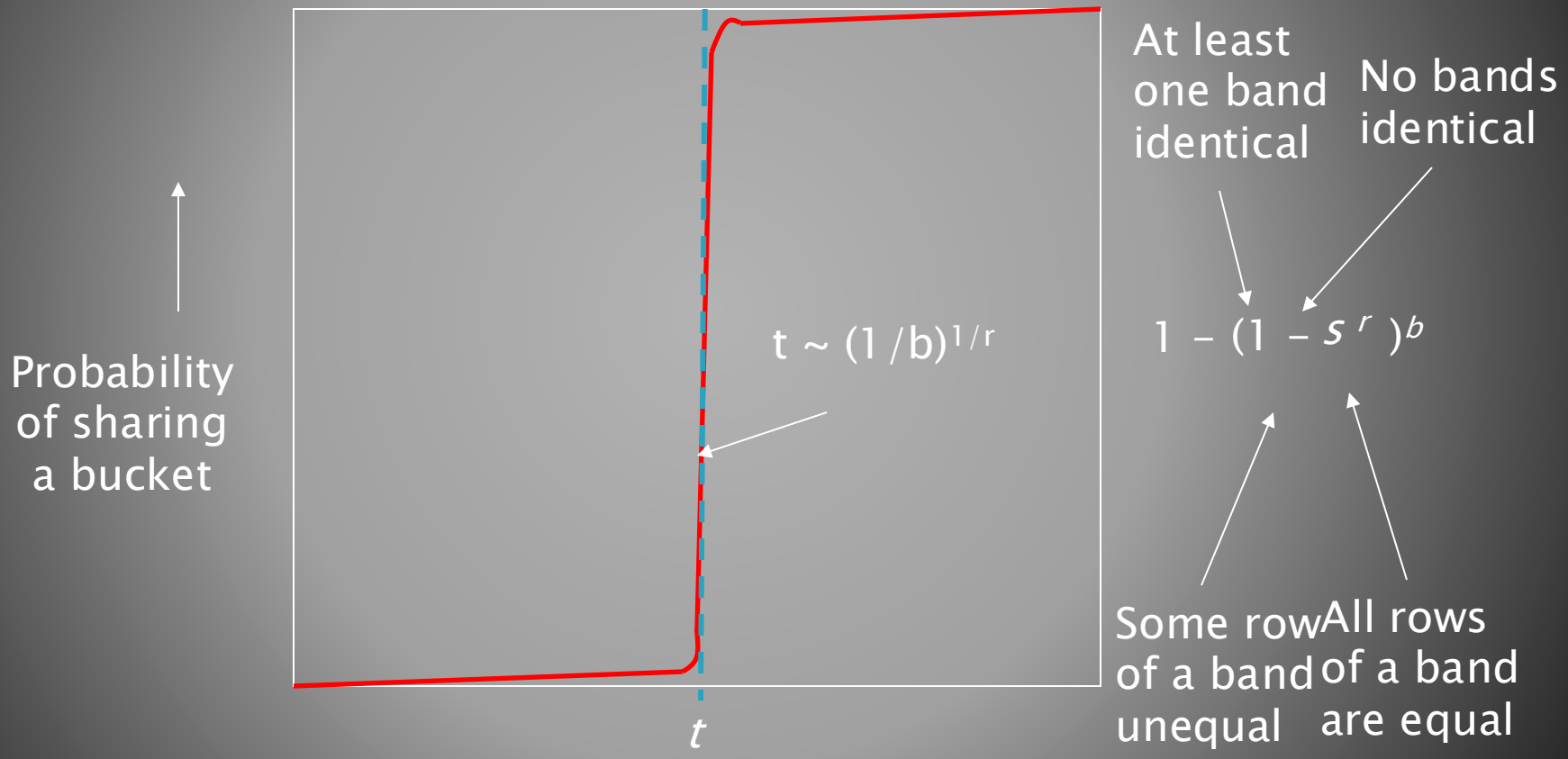


What One Band of One Row Gives You



Single hash signature

What *b* Bands of *r* Rows Gives You



Example: $b = 20$; $r = 5$

s	$1-(1-s^r)^b$
.2	.006
.3	.047
.4	.186
.5	.470
.6	.802
.7	.975
.8	.9996

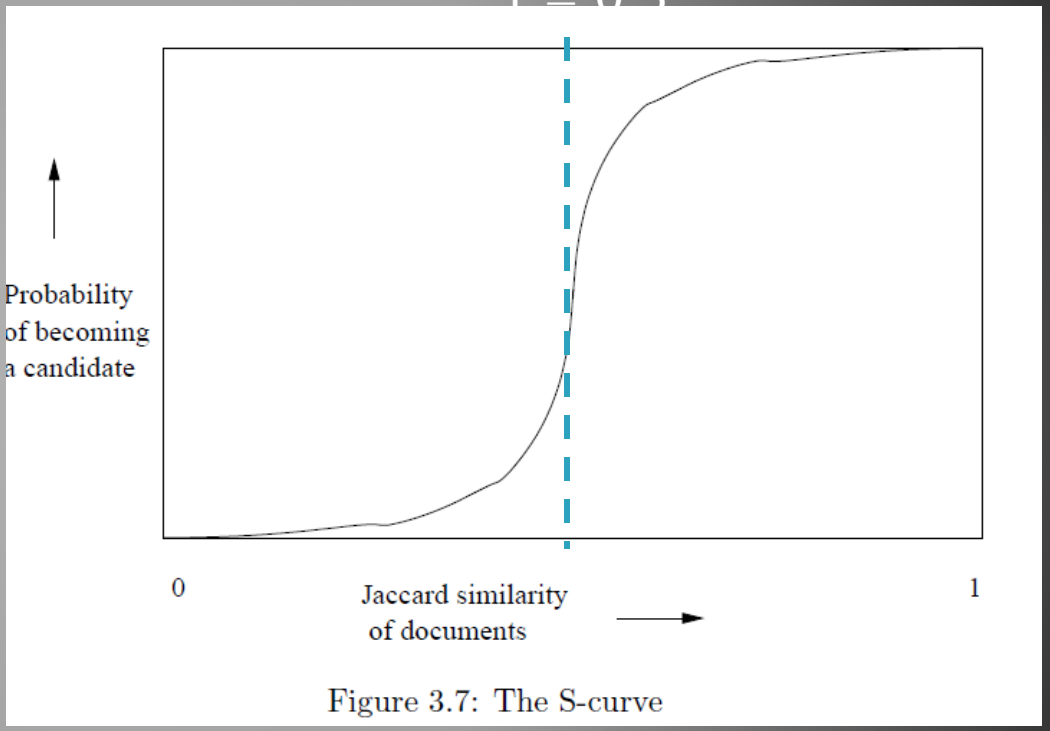


Figure 3.7: The S-curve

Suppose S_1, S_2 are 80% Similar

- We want all 80%-similar pairs. Choose 20 bands of 5 integers/band.
- Probability S_1, S_2 identical in one particular band:
$$(0.8)^5 = 0.328.$$
- Probability S_1, S_2 are not similar in any of the 20 bands:
$$(1-0.328)^{20} = 0.00035$$
  - i.e., about 1/3000-th of the 80%-similar column pairs are false negatives.
- Probability S_1, S_2 are similar in at least one of the 20 bands:
$$1-0.00035 = 0.999$$

C_1, C_2 are 30% Similar

- **Find pairs of ≥ *s*=0.8 similarity, set b=20, r=5**
- **Assume:** $\text{sim}(C_1, C_2) = 0.3$
  - Since $\text{sim}(C_1, C_2) <$ **s** we want C_1, C_2 to hash to **NO common buckets** (all bands should be different)
- **Probability C_1, C_2 identical in one particular band:** $(0.3)^5 = 0.00243$
- Probability C_1, C_2 identical in at least 1 of 20 bands: $1 - (1 - 0.00243)^{20} = 0.0474$
  - In other words, approximately 4.74% pairs of docs with similarity 0.3% end up becoming **candidate pairs**
    - They are **false positives** since we will have to examine them (they are candidate pairs) but then it will turn out their similarity is below threshold **s**

LSH Summary

- Tune to get almost all pairs with similar signatures, but eliminate most pairs that do not have similar signatures.
- Check in main memory that candidate pairs really do have similar signatures.
- Optional: In another pass through data, check that the remaining candidate pairs really represent similar *sets* .

Locality-sensitive hashing (LSH)

- Big Picture: Construct hash functions $h: R^d \rightarrow U$ such that for any pair of points p, q, for distance function D we have:
  - If $D(p,q) \leq r$, then $Pr[h(p)=h(q)] \geq \alpha$ is high
  - If $D(p,q) \geq cr$, then $Pr[h(p)=h(q)] \leq \beta$ is small
- Then, we can find close pairs by hashing

- LSH is a general framework: for a given distance function D we need to find the right h
  - h is (r, cr, α, β)-sensitive

Applications

- Entity Matching/Record Linkage
- Advertising Matching
- Image Search
- News Article
- Fingerprints